Taller práctico: Regresión logística paso a paso (con Python)

Fundamentos para IA · Semana 8 Versión estudiante

Dataset de trabajo: `StudentsPerformance.csv` (1000 registros). Este notebook desarrolla **en código Python** los 5 pasos y la autoevaluación de `02_Evaluacion_desarrollo.md`, que trae la explicación conceptual y los cálculos a mano de estos mismos ejercicios (sin resolver). Completa las celdas marcadas con `TODO` y responde las preguntas en las celdas de markdown indicadas.

Caso guía: `Y = 1` si el estudiante **aprueba matemáticas** (`math score >= 60`), `X = reading score`.

In [ ]:
```
import pandas as pd
import numpy as np
from scipy import stats
import statsmodels.api as sm
import matplotlib.pyplot as plt

df = pd.read_csv("StudentsPerformance.csv")
df.columns = [c.strip() for c in df.columns]
df["aprueba_mate"] = (df["math score"] >= 60).astype(int)
df.head()
```

Paso 1 — Del predictor lineal `z` a la probabilidad (sigmoide)

Concepto que refuerza: Preguntas 1 y 2 de la evaluación (el modelo es lineal en el *logit*, y la sigmoide convierte `z` en una probabilidad entre 0 y 1).

```
Paso A — predictor lineal:   z = b0 + b1 * X
Paso B — función sigmoide:   p(x) = 1 / (1 + e^(-z))
```

Primero ajustamos el modelo con `statsmodels` para obtener `b0` y `b1`.

In [ ]:
```
X = sm.add_constant(df["reading score"])
y = df["aprueba_mate"]

modelo = sm.Logit(y, X).fit(disp=0)
print(modelo.summary())

b0 = modelo.params["const"]
b1 = modelo.params["reading score"]
print(f"\nb0 = {b0:.4f}   b1 = {b1:.5f}")
```

Ejercicio 1.1. Completa la celda siguiente:

1. Implementa la función `sigmoide(z)`.

2. Calcula `z = b0 + b1 * reading score` para `reading score` = 40, 60, 70 y 90.
3. Calcula `p(x)` aplicando la sigmoide a `z`.

¿Qué observas en cómo cambia `z` frente a cómo cambia `p(x)` a medida que sube `reading score`?

In [ ]:
```
def sigmoide(z):
    # TODO: implementa 1 / (1 + e^(-z)) usando np.exp
    pass

tabla = pd.DataFrame({"reading score": [40, 60, 70, 90]})
tabla["z"] = None       # TODO: z = b0 + b1 * reading score
tabla["p(x)"] = None    # TODO: aplica sigmoide(z)
tabla
```

In [ ]:
```
# Curva sigmoide completa, para ver la forma en "S" (ejecuta esta celda después
xs = np.linspace(df["reading score"].min(), df["reading score"].max(), 200)
ps = sigmoide(b0 + b1 * xs)

plt.figure(figsize=(6, 4))
plt.plot(xs, ps, color="steelblue")
plt.scatter(df["reading score"], df["aprueba_mate"], alpha=0.15, color="gray")
plt.axhline(0.5, color="red", linestyle="--", linewidth=1)
plt.xlabel("reading score")
plt.ylabel("p(x) = P(aprueba matemáticas)")
plt.title("Curva sigmoide ajustada sobre los 1000 estudiantes")
plt.show()
```

Ejercicio 1.2 (verdadero/falso). "Si dos estudiantes A y B tienen el mismo `reading score`, siempre van a tener exactamente el mismo `p(x)`, sin importar ninguna otra información sobre ellos." ¿Verdadero o falso? Justifica.

Tu respuesta:

Paso 2 — Odds y odds ratio

Concepto que refuerza: Preguntas 3 y 4 de la evaluación (definición de *odds*, complemento de la probabilidad, e interpretación del coeficiente mediante el *odds ratio*).

```
odds = p / (1 - p)
```

In [ ]:
```
tabla["odds"] = None  # TODO: odds = p(x) / (1 - p(x))
tabla[["reading score", "z", "p(x)", "odds"]]
```

Ejercicio 2.1. Con la columna `odds` que acabas de calcular, ¿son mayores o menores a 1 los momios de aprobar para `reading score = 60`? ¿Y para `reading score = 70`? ¿Qué significa cada caso?

Tu respuesta:

In [ ]:
```
odds_ratio = None  # TODO: odds ratio = e^b1 (usa np.exp)
print(f"Odds ratio (OR = e^b1) = {odds_ratio:.4f}")
```

```
# Comprobación: subir reading score de 60 a 61 debería multiplicar los odds por
odds_60 = None  # TODO: calcula p y luego odds para reading score = 60
odds_61 = None  # TODO: calcula p y luego odds para reading score = 61
print(f"odds(60) = {odds_60:.4f}")
print(f"odds(61) = {odds_61:.4f}")
print(f"odds(60) * OR = {odds_60 * odds_ratio:.4f}  <- ¿se acerca a odds(61)?")
```

Ejercicio 2.2. En tus palabras, ¿qué significa el número que obtuviste para el *odds ratio*? Usa la comprobación de la celda anterior para explicarlo con tus propios términos.

Tu respuesta:

Paso 3 — El umbral de decisión y sus consecuencias

Concepto que refuerza: Parte de la Pregunta 4 (el rol del umbral) y las tarjetas de precisión/recall del material de teoría.

In [ ]:
```
for t in [0.5, 0.8]:
    tabla[f"clasificación (t={t})"] = None  # TODO: "Aprueba" si p(x) >= t, si n
    # pista: np.where(tabla["p(x)"] >= t, "Aprueba", "No aprueba")

tabla[["reading score", "p(x)", "clasificación (t=0.5)", "clasificación (t=0.8)"
```

Ejercicio 3.1. ¿A partir de qué `reading score` de la tabla se clasifica como "aprueba" con `t = 0.5`?

Ejercicio 3.2. Con `t = 0.8`, ¿qué le pasa al estudiante con `reading score = 70`? En general, ¿qué le pasa a la cantidad de estudiantes clasificados como "aprueba" cuando subes el umbral? ¿Por qué crees que pasa eso con la precisión y con el recall?

Tu respuesta:

Paso 4 — Evaluando el modelo: matriz de confusión

Concepto que refuerza: las tarjetas de "Precisión, Recall y umbral" de la teoría, y la idea de que *accuracy* puede engañar con clases desbalanceadas.

In [ ]:
```
p_todos = modelo.predict(X)
pred_todos = (p_todos >= 0.5).astype(int)

vp = None  # TODO: verdaderos positivos -> (pred_todos == 1) & (y == 1), y .sum(
vn = None  # TODO: verdaderos negativos
fp = None  # TODO: falsos positivos
fn = None  # TODO: falsos negativos

matriz = pd.DataFrame(
    [[vn, fp], [fn, vp]],
    index=["Real: no aprueba", "Real: aprueba"],
    columns=["Predicho: no aprueba", "Predicho: aprueba"],
```

```
)
matriz
```

In [ ]:
```
accuracy = None   # TODO: (vp + vn) / len(y)
precision = None  # TODO: vp / (vp + fp)
recall = None     # TODO: vp / (vp + fn)

print(f"Exactitud (accuracy) = {accuracy:.4f} -> {accuracy:.1%}")
print(f"Precisión            = {precision:.4f} -> {precision:.1%}")
print(f"Recall (sensibilidad)= {recall:.4f} -> {recall:.1%}")

# Modelo "tonto": siempre predice "aprueba". ¿Qué accuracy tendría?
accuracy_tonto = None  # TODO: proporción de estudiantes que sí aprueban (y.mean
print(f"\nAccuracy de un modelo 'tonto' que siempre predice aprueba: {accuracy_t
```

Ejercicio 4.1. Con los resultados anteriores, ¿el modelo es mejor detectando a los que sí aprueban (recall) o siendo preciso cuando dice "aprueba" (precisión)?

Ejercicio 4.2. Compara la *accuracy* del modelo real contra la del modelo "tonto". ¿Por qué no basta con mirar la *accuracy* sola para decidir si un modelo de clasificación es bueno, sobre todo cuando las clases están desbalanceadas (677 aprueban vs. 323 no aprueban)?

Tu respuesta:

Paso 5 — Estadística inferencial: ¿el efecto es real o es azar?

Concepto que refuerza: Pregunta 5 de la evaluación (distinguir el modelado de la validación estadística: inferencia, confianza, significancia, colinealidad).

- **H0:** el verdadero coeficiente de `reading score` es cero (no influye).
- **H1:** el verdadero coeficiente es distinto de cero (sí influye).
- Prueba de Wald: usa el estadístico *z* y su p-value, ya calculados en el `summary()` del Paso 1.

In [ ]:
```
z_wald = None     # TODO: modelo.tvalues["reading score"]
p_value = None    # TODO: modelo.pvalues["reading score"]
ic = None         # TODO: modelo.conf_int().loc["reading score"]

print(f"Estadístico z (Wald) = {z_wald:.2f}")
print(f"p-value              = {p_value:.4g}")
print(f"Intervalo de confianza 95% de b1 = ({ic[0]:.4f}, {ic[1]:.4f})")

# Colinealidad: correlación entre dos posibles predictoras
correlacion = None  # TODO: df[["reading score", "writing score"]].corr().iloc[0
print(f"\nCorrelación reading score vs writing score = {correlacion:.3f}  (ejemp
```

Ejercicio 5.1. Con base en la salida anterior y en la teoría, clasifica cada uno de estos términos como parte del **modelado** o de la **validación estadística**, completando la tabla:

Término	Modelado o validación estadística
sigmoide	
p-value	
odds ratio	
intervalo de confianza	
umbral	
colinealidad	
z (predictor lineal)	

Ejercicio 5.2. Un estudiante afirma: "Como el p-value de `reading score` es prácticamente cero, eso significa que `reading score` explica el 100% de si un estudiante aprueba matemáticas o no." ¿Estás de acuerdo? Justifica usando lo que calculaste en el Paso 4.

Tu respuesta:

Autoevaluación final

Antes de presentar "MA Evaluación 8", responde estas preguntas con tus propias palabras (sin mirar el material):

1. ¿Por qué la regresión logística se dice "lineal en el logit" y no "lineal en la probabilidad"?
2. Escribe la fórmula de la sigmoide de memoria y explica qué representa cada símbolo (`z`, `e`, `p(x)`).
3. Define *odds* con tus propias palabras y calcula los *odds* de un evento con `p = 0.75` en la celda de código siguiente.
4. Explica para qué sirve el odds ratio y qué significaría un `OR = 2`.
5. Da un ejemplo (distinto al de este taller) de un concepto de **modelado** y uno de **validación estadística** en regresión logística.

Tus respuestas:

In [ ]:
```
# Ejercicio 3 de la autoevaluación: calcula los odds para p = 0.75
p_ejemplo = 0.75
odds_ejemplo = None  # TODO
print(f"odds = {odds_ejemplo:.2f}")
```

Material relacionado

Explicación conceptual y cálculos a mano (sin código): `02_Evaluacion_desarrollo.md`. Clave con explicación y respuestas resueltas (código y celdas completas): `02_Evaluacion_taller_profesor.ipynb`. Clave de las 5 preguntas de la evaluación real: `02_Evaluacion_8_clave_respuestas_profesor.md`.

Teoría de regresión logística: `01_Regresion_logistica_conceptos_basicos.md`.
Comparación con regresión lineal: `03_Regresion_Lineal_Vs_Regresión Logística.md`.